

Programmation par Contraintes

Spatially Balanced Latin Square (SBLS)

Auteur :
Bensallah Younes

Université Côte d'Azur
Master 1 Informatique
Année universitaire 2025–2026
Date de rendu : 25 Janvier 2026

Dépôt GitHub : `Spatially Balanced Latin Square (SBLS)`

Table des matières

1 Introduction **2**

2 Modélisation **2**
2.1 Variables de décision 2
2.2 Contrainte 1 : Carré latin 2
2.3 Contrainte 2 : Orthogonalité 2
2.4 Contrainte 3 : Équilibre spatial 2
2.5 Cassage de symétries 3

3 Stratégie de recherche **3**
3.1 Modèle Simple 3
3.2 Modèle Utilisé 3

4 Résultats expérimentaux **3**
4.1 Analyse 3

5 Conclusion **4**
5.1 Conclusion et Améliorations possibles 4
5.2 Utilisation de l'IA 4

1 Introduction

Le problème des SBLS s'applique dans la conception d'expériences agricoles. L'objectif est alors de trouver deux carrés latins A et B de taille $n \times n$ satisfaisant les trois propriétés ci-dessous :

1. **Carré latin** : chaque symbole $0, \ldots, n-1$ apparaît exactement une fois par ligne et par colonne
2. **Orthogonalité** : chaque paire $(A_{i,j}, B_{i,j})$ est unique
3. **Équilibre spatial** : pour toute paire de symboles (p, q), la somme des distances au carré entre les cellules contenant p et celles contenant q est identique,

2 Modélisation

2.1 Variables de décision

Pour chaque cellule (i, j) avec $i, j \in \{0, \ldots, n-1\}$:
— $A_{i,j} \in \{0, \ldots, n-1\}$: valeur de la matrice A en position (i, j)
— $B_{i,j} \in \{0, \ldots, n-1\}$: valeur de la matrice B en position (i, j)

2.2 Contrainte 1 : Carré latin

Pour chaque ligne i et chaque colonne j :

$$\text{allDifferent}(A_{i,0}, A_{i,1}, \ldots, A_{i,n-1}) \tag{1}$$
$$\text{allDifferent}(A_{0,j}, A_{1,j}, \ldots, A_{n-1,j}) \tag{2}$$

Idem pour la matrice B. Le filtrage utilisé est **AC** (Arc Consistency).

2.3 Contrainte 2 : Orthogonalité

On encode chaque paire comme une valeur unique via la contrainte `scalar` :

$$P_{i,j} = n \cdot A_{i,j} + B_{i,j} \tag{3}$$

Puis on impose :

$$\text{allDifferent}(P_{0,0}, P_{0,1}, \ldots, P_{n-1,n-1}) \tag{4}$$

2.4 Contrainte 3 : Équilibre spatial

Pour chaque paire de symboles (p, q) avec $p < q$, on définit la somme des distances :

$$D^A_{p,q} = \sum_{\substack{(i_1,j_1),(i_2,j_2) \\ A_{i_1,j_1}=p, A_{i_2,j_2}=q}} \left[(i_1 - i_2)^2 + (j_1 - j_2)^2\right] \tag{5}$$

L'équilibre spatial impose que toutes ces sommes soient égales :

$$D^A_{0,1} = D^A_{0,2} = \cdots = D^A_{n-2,n-1} \tag{6}$$

2.5 Cassage de symétries

Pour réduire l'espace de recherche on casse la symétrie :
— $A_{0,0} = 0$ et $B_{0,0} = 0$
— Première ligne de A fixée : $A_{0,j} = j$
— Première colonne de A fixée : $A_{i,0} = i$

3 Stratégie de recherche

3.1 Modèle Simple

— **Sélection de variable** : InputOrder (ordre lexicographique)
— **Sélection de valeur** : IntDomainMin (plus petite valeur)

3.2 Modèle Utilisé

— **Sélection de variable** : DomOverWDeg (dom/wdeg)
  — Choisit la variable avec le plus petit ratio domaine/poids
  — Les poids augmentent quand une contrainte cause un échec
  — Apprentissage adaptatif pendant la recherche
— **Sélection de valeur** : IntDomainMin

4 Résultats expérimentaux

Les tests ont été effectués sur une machine avec Processeur AMD Ryzen 7 3700X, 16 cœurs, 16 Go de RAM.

n	Simple			Notre méthode		
	Temps (ms)	Nœuds	Backtracks	Temps (ms)	Nœuds	Backtracks
3	8	4	0	18	2	0
4	11	10	0	141	24	37
5	590	6735	13438	1373	327	637

TABLE 1 – Comparaison des performances

4.1 Analyse

— Le modèle simple est plus rapide car il n'a pas la contrainte d'équilibre spatial
— Notre méthode explore moins de nœuds grâce à DomOverWDeg qui guide mieux la recherche
— Pour $n = 5$, notre méthode visite 327 nœuds contre 6735 pour le modèle simple
— Les backtracks sont plus nombreux dans le simple (13438 vs 637) car il explore plus de branches sans succès
— Pour $n \geq 6$, la contrainte d'équilibre spatial génère $O(n^4)$ termes, rendant la résolution difficile

5 Conclusion

5.1 Conclusion et Améliorations possibles

Nous avons présenté une modélisation CP du problème SBLS utilisant Choco Solver. Notre approche :

— Utilise des contraintes globales efficaces (allDifferent avec GAC)
— Implémente l'équilibre spatial via réification et contraintes arithmétiques
— Emploie la stratégie DomOverWDeg pour améliorer l'exploration

Le modèle résout efficacement les instances jusqu'à $n = 5$. Pour des instances plus grandes, des techniques comme LNS (Large Neighborhood Search) seraient plus judicieuses.

5.2 Utilisation de l'IA

L'IA a été utilisée dans les points suivants :

— Recherche et utilisation de stratégies plus adaptées, notamment depuis la documentation choco
— Aide pour l'ajout et utilisation de la librairie Choco dans le projet
— Simplification des scripts de test.

Références

1. Cours Régin, J.-C. Introduction Programmation Par Contraintes.
2. Documentation Choco Solver : `https://choco-solver.org/`